Knowledge-base import autotests

Tests for the knowledge-base import that runs on container startup (PR #168): the `scripts/import_knowledge.sh` gating logic and the entrypoint wiring that triggers it.

- `test_import_knowledge.py` — unit tests for `scripts/import_knowledge.sh`, with the `import-knowledge` binary stubbed on PATH.
- `test_import_knowledge_integration.py` — startup tests that launch the image through `scripts/omega` and check the entrypoint gating and that a local import lands in the `chroma_db` the agent reads.

Build the image

The integration tests need a locally built image. Build it from the repository root:

```
docker build -t omega:kb-test .
```

The unit tests need no image.

Run

Unit:

```
cd Autotests
pytest -s import_knowledge/test_import_knowledge.py
```

Integration (point `OMEGA_KB_IMAGE` at the image built above):

```
cd Autotests
OMEGA_KB_IMAGE=omega:kb-test pytest -s import_knowledge/test_import_knowledge_integration.py
```

The integration tests start and remove the container themselves through `scripts/omega` (fixed name `omega`, one at a time), so there is no manual `docker run` step. They are skipped unless `OMEGA_KB_IMAGE` is set and Docker and `scripts/omega` are available.

Tests description

Unit: scripts/import_knowledge.sh

The script reads `EMBEDDING_PROVIDER`, runs `import-knowledge` (or `import-knowledge --local`), and writes a per-provider sentinel under `CHROMA_DB_PATH` so a second start skips the import. The stub on PATH records how `import-knowledge` was called.

1. test_local_runs_import_and_writes_sentinel

Local provider runs the import and leaves the local sentinel behind.

- Checks: exit 0; the stub was called with `--local`; `.import-kb.local.done` exists; stdout reports `Import complete`.

2. test_openai_with_key_runs_import_and_writes_sentinel

OpenAI with `OPENAI_API_KEY` set runs the import and leaves the OpenAI sentinel.

- Checks: exit 0; the stub was called without `--local`; `.import-kb.openai.done` exists.

3. test_openai_without_key_exit1

OpenAI without a key stops before importing.

- Checks: exit 1; stderr says `OPENAI_API_KEY is required`; the stub was never called.

4. test_unknown_provider_exit1

A provider the script doesn't know is rejected.

- Checks: exit 1; stderr says `Unsupported`; the stub was never called.

5. test_provider_case_insensitive

The provider name is matched regardless of case (`openai` / `OpenAI` / `OPENAI`, `local` / `Local` / `LOCAL`).

- Checks: exit 0; the sentinel for the matching provider is written. Parametrized over the six spellings.

6. test_existing_sentinel_skips_import

A sentinel already present means the import is skipped.

- Checks: exit 0; stdout says `skipping`; the stub was never called.

7. test_force_reimports_despite_sentinel

`IMPORT_KB_FORCE=1` re-runs the import even with the sentinel present.

- Checks: exit 0; the stub was called with `--local`.

8. test_local_openai_sentinels_independent

Local and OpenAI track separate sentinels, so one being done doesn't block the other.

- Checks: with `.import-kb.local.done` already present, OpenAI still imports and writes `.import-kb.openai.done`.

9. test_custom_chroma_path

`CHROMA_DB_PATH` sends the DB directory and its sentinel to a custom location.

- Checks: the custom directory is created; the sentinel lands inside it.

10. test_failed_import_does_not_write_sentinel

When `import-knowledge` exits non-zero, no sentinel is written, so the next start retries instead of masking the failure.

- Checks: non-zero exit; the stub was called; `.import-kb.local.done` is absent.

Integration: container startup

They launch the image through `scripts/omega`, so the real entrypoint runs (nginx, env scrub, import gating).

11. test_entrypoint_imports_when_enabled

With `IMPORT_KB_ON_START=1` the entrypoint starts the import on boot.

- Checks: `[import-kb] Running` appears in the container log within 180 s.

12. test_entrypoint_skips_when_disabled

With `IMPORT_KB_ON_START=0` the entrypoint never touches import-kb.

- Checks: after a 25 s window no `[import-kb]` line appears in the log.

13. test_local_real_import_runs

A real local import runs and lands in the same `chroma_db` the agent reads from.

- Checks: the log shows `Running import-knowledge with local embeddings` and `Generating local`; inside the container `/PeTTa/chroma_db` resolves to `$MEMORY_DIR/chroma_db`, so the import target is the path the agent queries.